

42. generate this pattern

* *

* *

* *

Algorithm

Step 1: Start the program.

Step 2: Initialize a loop variable i from 0 to 3 (for 4 rows).

Step 3: For each iteration of the loop:

 If i is 0 (i.e., the first row), print ****.

 Else, print * * (a star, two spaces, and a star).

Step 4: Repeat until all 4 rows are printed.

Step 5: End the program.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    for (int i = 0; i < 4; i++) {
```

```
        if (i == 0) {
```

```
            printf("****\n");
```

```
        } else {
```

```
            printf(" * \n");
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Output:

```
****
*  *
*  *
*  *
*  *

[1] + Done          "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$
```

43. generate this pattern

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Algorithm

Step 1: Start the program.

Step 2: Use an outer loop from $i = 1$ to 5 (each line number).

Step 3: Inside the outer loop, use an inner loop from $j = 1$ to i .

Step 4: Print the value of j followed by a space.

Step 5: After the inner loop ends, print a newline.

Step 6: Repeat until 5 rows are printed.

Step 7: End the program.

Code:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }
    return 0;}
```

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

[1] + Done                                "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$

```

Output:

44. generate this pattern

1 2 3 4 5

1 2 3 4

1 2 3

1 2

1

Algorithm:

Step 1: Start the program.

Step 2: Use a loop from $i = 5$ to 1.

Step 3: For each i , run an inner loop from $j = 1$ to i .

Step 4: Print j with a space.

Step 5: Print a newline after the inner loop.

Step 6: End the program.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int i, j, num;
```

```
    for (i = 0; i < 5; i++) {
```

```
        num = 1;
```

```
        for (j = 0; j <= i; j++) {
```

```
            printf("%d ", num);
```

```

        num = num * (i - j) / (j + 1);
    }
    printf("\n");
}
return 0;
}

```

Output:

```

1 2 3 4 5
1 2 3 4
1 2 3
1 2
1

[1] + Done                               "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp/
mamata-rai@Mamata-Rai:~$

```

45. generate this pattern

```

1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

```

Algorithm:

Step 1: Start program.

Step 2: Loop through rows $i = 0$ to 4.

Step 3: Set $\text{num} = 1$.

Step 4: Loop $j = 0$ to i , print num .

Step 5: Update $\text{num} = \text{num} * (i - j) / (j + 1)$.

Step 6: Print newline after each row.

Code:

```
#include <stdio.h>

int main() {
    int i, j, num;
    for (i = 0; i < 5; i++) {
        num = 1;
        for (j = 0; j <= i; j++) {
            printf("%d ", num);
            num = num * (i - j) / (j + 1);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

[1] + Done          "/usr/bin/gdb" --interpreter=mi -tty=${DbgTerm} ...
mamata-rai@Mamata-Rai:~$
```

46. generate this pattern

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Algorithm:

Step 1: Initialize count = 1.

Step 2: Loop i = 1 to 5 (rows).

Step 3: For each row, print i numbers starting from count.

Step 4: Increment count.

Code:

```
#include <stdio.h>

int main() {
    int count = 1;
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d ", count++);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15

[1] + Done ...
mamata-rai@Mamata-Rai:~$
```

47. generate this pattern

```
*   *
*   *
*   *
*
*   *
*   *
```

* *

Algorithm:

Step 1: Start the program.

Step 2: Set $n = 4$ (middle row index).

Step 3: Loop from $i = 0$ to $2*n - 2$ (i.e., 0 to 6).

Step 4: For each row, calculate number of spaces before the first *.

Step 5: Print a *, then inner spaces, then another * (except middle row has only one *).

Step 6: Move to next line.

Step 7: End program.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n = 4;
```

```
    int totalRows = 2 * n - 1;
```

```
    for (int i = 0; i < totalRows; i++) {
```

```
        int spaces = i < n ? i : totalRows - i - 1;
```

```
        int innerSpaces = (n - 1 - spaces) * 2 - 1;
```

```
        for (int j = 0; j < spaces; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        printf("*");
```

```
        if (innerSpaces >= 1) {
```

```
            for (int j = 0; j < innerSpaces; j++) {
```

```
                printf(" ");
```

```
            }
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
}
```

Output:

```
*      *
*      *
*      *
*      *
*      *
*      *
*      *
*      *
*      *

[1] + Done          "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$
```

48. generate this pattern

```
*****

***

*

***
```

Algorithm:

Step 1: Create array: {5, 3, 1, 3}

Step 2: Loop over each number and print that many stars.

Code:

```
#include <stdio.h>

int main() {

    int lines[] = {5, 3, 1, 3};

    for (int i = 0; i < 4; i++) {

        for (int j = 0; j < lines[i]; j++) {
```



```

        printf("*");

    }

    printf("\n");

}

return 0;

}

```

Output:

```

*****
***
*
***

[1] + Done ...
mamata-rai@Mamata-Rai:~$

```

49. generate this pattern

```

* * * *
* * *
* *
*
* *
* * *
* * * *

```

Algorithm:

Step 1: Start the program.

Step 2: Set $n = 4$ (total rows in half).

Step 3: Loop from $i = 0$ to $2*n - 1$ (i.e., 0 to 7 exclusive).

Step 4: For the upper part ($i < n$), increase spaces and decrease stars.

Step 5: For the lower part ($i \geq n$), decrease spaces and increase stars.

Step 6: Use nested loops to print spaces and * pattern.

Step 7: Move to the next line after each row.

Step 8: End the program.

Code:

```
#include <stdio.h>

int main() {

    int n = 4;

    for (int i = 0; i < 2 * n - 1; i++) {

        int spaces = i < n ? i : 2 * n - 2 - i;

        int stars = i < n ? n - i : i - n + 2;

        for (int s = 0; s < spaces; s++) {

            printf(" ");

        }

        for (int j = 0; j < stars; j++) {

            printf("* ");

        }

        printf("\n");

    }

    return 0;

}
```

Output:

```
* * * *
* * *
* *
*
* *
* * *
* * * *
```

[1] + Done "/usr/bin/gdb" --interpreter=mi --tty=\${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~\$

50. generate this pattern

```
*
**
***
****
*****
*****
****
***
**
*
```

Algorithm:

Step 1: Loop i = 1 to 5, print i stars.

Step 2: Loop i = 4 to 1, print i stars.

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    for (int i = 1; i <= 5; i++) {
```

```

        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    for (int i = 4; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}

```

Output:

```

*
**
***
****
*****
****
***
**
*

[1] + Done ...
mamata-rai@Mamata-Rai:~$

```

51. Write a program that reads integers from the user until a zero is entered. Use a while loop to read the integers. Use break to exit the loop when zero is entered. Calculate and print the sum of all the entered numbers.

Algorithm:

Step 1: Start

Step 2: Initialize sum = 0

Step 3: Loop forever

 Read a number

 If number is 0, break the loop

 Else, add it to sum

Step 4: Print the total sum

Step 5: End

Code:

```
#include <stdio.h>

int main() {

    int num, sum = 0;

    while (1) {

        printf("Enter a number (0 to stop): ");

        scanf("%d", &num);

        if (num == 0)

            break;

        sum += num;

    }

    printf("Sum = %d\n", sum);

    return 0;

}
```

Output:

```
Enter a number (0 to stop): 10
Enter a number (0 to stop): 20
Enter a number (0 to stop): 30
Enter a number (0 to stop): 0
Sum = 60

[1] + Done                                "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$
```

52. Write a program that reads 10 integers from the user. Use a for loop to read the integers. Use continue to skip the processing of any negative numbers. Calculate and print the sum of all the positive numbers.

Algorithm:

Step 1: Start

Step 2: Initialize sum = 0

Step 3: Loop 10 times

 Read a number

 If number < 0, use continue

 Else, add to sum

Step 4: Print the sum

Step 5: End

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int num, sum = 0;
```

```
    for (int i = 0; i < 10; i++) {
```

```

    printf("Enter number %d: ", i + 1);

    scanf("%d", &num);

    if (num < 0)

        continue;

    sum += num;

}

printf("Sum of positive numbers = %d\n", sum);

return 0;

}

```

Output:

```

Enter number 1: 5
Enter number 2: -2
Enter number 3: 3
Enter number 4: 7
Enter number 5: -9
Enter number 6: 1
Enter number 7: 2
Enter number 8: 6
Enter number 9: -3
Enter number 10: 8
Sum of positive numbers = 32

[1] + Done                                "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$

```

53. Write a program to print a multiplication table for numbers from 1 to 5. Use nested for loops to generate the table. Use continue to skip printing the multiples of 3.

Algorithm:

Step 1: Start

Step 2: Loop from $i = 1$ to 5

Loop from j = 1 to 10

if i*j is multiple of 3, continue

Print i * j

Step 3: End

Code:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        printf("Table of %d:\n", i);
        for (int j = 1; j <= 10; j++) {
            int product = i * j;
            if (product % 3 == 0)
                continue;

            printf("%d x %d = %d\n", i, j, product);
        }
        printf("\n");
    }
    return 0;
}
```

Output:


```

Table of 1:
1 x 1 = 1
1 x 2 = 2
1 x 4 = 4
...

Table of 3:
3 x 2 = 6 ← skipped
3 x 3 = 9 ← skipped
...

[1] + Done                                "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$

```

54. Write a program that reads a number from the user. Use if-else statements and loops to determine if the number is prime. Implement the logic without using the goto statement. Briefly explain why it's generally considered better to avoid goto statements.

Algorithm:

Step 1: Start

Step 2: Read a number

Step 3: If number < 2 → Not prime

Step 4: Loop from i = 2 to sqrt(number)

If divisible, set flag to not prime

Step 5: Print result

Step 6: End

Code:

```

#include <stdio.h>

int main() {
    int num, isPrime = 1;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num < 2)

```

```

        isPrime = 0;
    else {
        for (int i = 2; i <= num / 2; i++) {
            if (num % i == 0) {
                isPrime = 0;
                break;
            }
        }
    }

    if (isPrime)
        printf("%d is a prime number\n", num);
    else
        printf("%d is not a prime number\n", num);

    return 0;
}

```

Output:

```

Enter a number: 13
13 is a prime number

[1] + Done                                "/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,
mamata-rai@Mamata-Rai:~$

```

dcks

sdvsd

55. Write a menu driven program according to user choice, if user press

1. Calculate factorial of a number
2. Find the sum of natural numbers

3. Check for prime number
4. Calculate area of a circle
5. Calculate power of a number
6. Exit

Algorithm:

Step 1: Start

Step 2: Display menu options

Step 3: Read user choice

Step 4: Use switch-case to call:

Case 1: Factorial

Case 2: Sum of natural numbers

Case 3: Prime check

Case 4: Area of circle

Case 5: Power of number

Case 6: Exit

Step 5: End

Code:

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    int choice, n, i, fact = 1, sum = 0, isPrime = 1;
```

```
    float radius, area, base, power;
```

```
    printf("Menu:\n");
```

```
    printf("1. Factorial\n2. Sum of natural numbers\n3. Prime check\n4. Area of circle\n5. Power\n6. Exit\n");
```

```
    printf("Enter your choice: ");
```

```
scanf("%d", &choice);
```

```
switch (choice) {
```

```
    case 1:
```

```
        printf("Enter number: ");
```

```
        scanf("%d", &n);
```

```
        for (i = 1; i <= n; i++) fact *= i;
```

```
        printf("Factorial = %d\n", fact);
```

```
        break;
```

```
    case 2:
```

```
        printf("Enter number: ");
```

```
        scanf("%d", &n);
```

```
        sum = n * (n + 1) / 2;
```

```
        printf("Sum = %d\n", sum);
```

```
        break;
```

```
    case 3:
```

```
        printf("Enter number: ");
```

```
        scanf("%d", &n);
```

```
        if (n < 2) isPrime = 0;
```

```
        else {
```

```
            for (i = 2; i <= n / 2; i++)
```

```
                if (n % i == 0) { isPrime = 0; break; }
```

```
        }
```

```
        printf(isPrime ? "Prime\n" : "Not Prime\n");
```

```
        break;
```

```
    case 4:
```

```
        printf("Enter radius: ");
```

```
    scanf("%f", &radius);  
    area = 3.14 * radius * radius;  
    printf("Area = %.2f\n", area);  
    break;  
case 5:  
    printf("Enter base and exponent: ");  
    scanf("%f %f", &base, &power);  
    printf("Result = %.2f\n", pow(base, power));  
    break;  
case 6:  
    printf("Exiting...\n");  
    break;  
default:  
    printf("Invalid choice\n");  
}  
return 0;  
}
```

Output:

Menu:

1. Factorial
2. Sum of natural numbers
3. Prime check
4. Area of circle
5. Power
6. Exit

Enter your choice: 1

Enter number: 5

Factorial = 120

[1] + Done

`"/usr/bin/gdb" --interpreter=mi --tty=${DbgTerm} 0<"/tmp,`

mamata-rai@Mamata-Rai:~\$